

AI Toolbox for Empirical Research

Use cases from six months of doing economics with Claude Code

Faqiang Li
faqiangli.econ@gmail.com

Peking University

September 18, 2026

Draws on Paul Goldsmith-Pinkham's *Getting Started with Claude Code* (substack + video series)
and on about 190 of my own working sessions with Claude Code, March–September 2026.

Share and adapt with credit, not for commercial use, under the same terms.

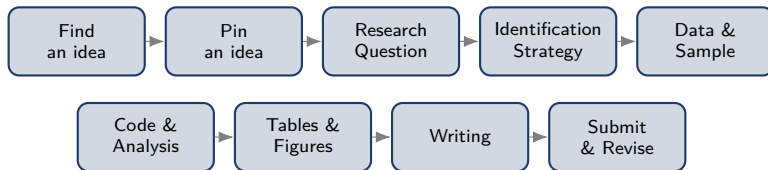
How to use this deck

- ① Please don't sell my slide and tips for money, but do feel free to share to your friends.
- ② Please know now the limit of the slide.

Share and adapt with credit, not for commercial use, under the same terms.

Every prompt I show today is yours to copy. A companion document with the prompts and the conversations will follow.

The same pipeline, now with an AI agent in the room



Today: the data-to-writing stretch.

Today's roadmap (two hours, one break)

Hour 1 — knowing Claude Code

- a. **Where I am coming from** — six months of use, so you can be appropriately skeptical.
- b. **Claude Code basics, fast and live** — the words, my daily routine, the context window, what a session really is; then writing the prompt, managing one session, and managing work across sessions.
- c. **Hacks worth stealing** — one-liners first, then small prompts, each shown as task, prompt and effect.
- d. **Collecting data at scale** — one full operation, with Claude and DeepSeek doing different jobs.
- e. **How to improve** — two rhythms, two prompts.

Break

Hour 2 — two structural projects, shown live

- ▷ Long live session: the Wisconsin project.
- ▷ Long live session: the solar project.

If you remember only one thing from today, make it the context window. Everything else sits on top of that one idea.

Part a

Where I am coming from

Part a: Where I am coming from

Six things about me (so think with caution when taking my advices in using AI)

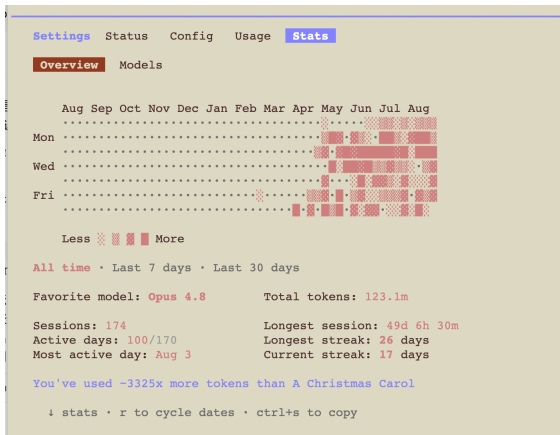
- ① 2 years as Assistant Professor at PKU.
- ② Empirical IO economist.
- ③ Communicated with **20+ RAs** over the years.
 - My style as a PI: lengthy, detailed, sometimes inefficient requirements — surprisingly compatible with what good context engineering turns out to require.
- ④ ~2 years of chat-based AI (ChatGPT, then Claude).
- ⑤ ~6 months of agentic AI coding (Claude Code): about 190 working sessions and 86 long unattended batch runs, March–September 2026.
- ⑥ **I pay \$200 a month, and in most weeks I use nearly all of it.**
 - So a lot of what I show today is priced at that level. **Scale it to what you are comfortable paying** — almost every habit here works on a smaller plan, just with fewer parallel workers and shorter unattended runs.

Six months of pushing these tools on real research, not three weeks of YouTube. Much of what follows is still a working hypothesis. **Push back when it does not make sense.**

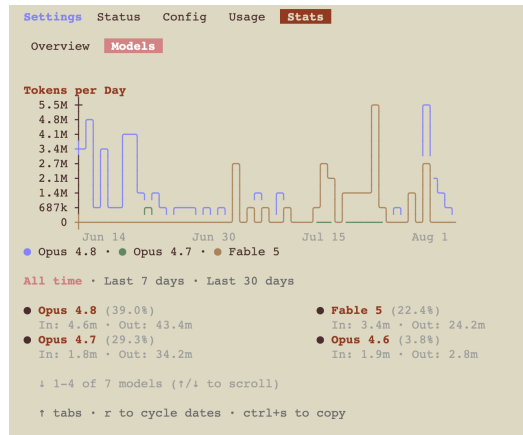
What I want these two hours to do

1. Knowledge diffusion.
2. Learn from you (some of you know things I do not).
3. Take stock of my own practice: preparing this talk made me read back through all six months of my sessions.

Part a: Six months, in numbers



/stats: activity by day, sessions, tokens



/stats, models tab: tokens per day by model

A reference point, not a boast: judge what follows against how much I have actually used it.

Part a: Opinions on what AI can do really well (my view, today)

- ① **Be your customised Stack Overflow / wiki / literature feed.**
- ② **Derive algebra.** Long derivations, checked step by step, with numbers substituted in to catch the slips.
- ③ **Replicate and adapt others' scripts.** Published code becomes a starting point you can edit in conversation, not a black box.
- ④ **Extensive web search and scraping.** The boring data-collection lift is dramatically lower.
- ⑤ **Test an idea quickly.** From hunch to first result in an afternoon, not a fortnight.
- ⑥ **The full workflow collapses.** Idea → data cleaning → analysis → documentation runs end-to-end fast. Project *iteration* is the biggest win — you can afford to try things you would not have tried before.

The last one is what today is mostly about.

Part a: Opinions on what AI cannot (yet) do — what stays on you

- ① **Thinking.** Think through ideas yourself *first*. Use AI to clarify and concretise, not to generate the idea itself. The right question is “*what am I missing or wrong about?*” — not “*what should I think?*”
- ② **Your idea, network, resources, taste.** These are what bind a good empirical project. AI helps least with these, and they remain yours to develop.
- ③ **Identification and the story.** Sketch the identification argument yourself first — then ask AI to help find the data sources that match that argument.
- ④ **The full-arc view of a project** (idea → submission → revision). Don't delegate the whole arc. The judgement calls that compound across the arc are yours.

Part a: My feelings on agentic AI tools, six months in

- ① **Good: it has not killed the fun part.** The fun of empirical work is the discovery, the design, the surprise. If you ask AI to write the paper end-to-end, you skip that. Don't. (Open normative question: *do* we actually need the fun, or is that just romance?)
- ② **I work more hours now — and I look forward to work.** The fun part is bigger. The boring-but-important part (cleaning, debugging, documenting) is mostly delegated. Net effect: more good hours per week.
- ③ **I love agentic coding because I love writing detailed work plans for students.** The same impulse that produces good RA briefs produces good Claude Code prompts. The asymmetry: the agent never tires of long, detailed instructions.
- ④ **My habit of giving RAs a great amount of detail and context helps a lot in context engineering.**
 - Getting good results from AI is mostly **asking a good question with enough, precise context.**
 - It is the same act as briefing an RA for a week: **predict the problems they will hit, and answer them up front.**
 - Where the files are; which results to build on; where the task sits in the big picture; what *not* to do.

Three of my actual RA work plans follow — as visual evidence, not as content to read.

20260127 new structural — Saved to my Mac

Home Insert Draw Design Layout References Mailings Review >> Comments Editing Share

Paste Aptos (Body) 12 A⁺ A⁻ A_a A_b Paragraph Styles Dictate Add-ins Editor Copilot

actually not so appropriate. I will explain this below in the sample part.

2. Sample

- We are only using one year, 2019 for running the demand estimation. This is the year when we get the highest quality of data. It is fine. There is no need to use multiple years to estimate this demand.
- We constrain in 10 cities where there are the most Venezuelans. Lima are regarded as four cities.

3. Markets

- Here is the tricky part. Currently, all students are assumed to be in one market. This is not typically done in school choice literature. Because students who are suppose to be in grade 1 are going to apply only to grade 1. So different grades should be regarded as separate markets. Besides, in the reduced form part, we are using within-school across-grade variation (namely, the "cohort variation" in the peer effect literature) to identify the exposure to Venezuelans. So there exists important variation across grades

market and instead regarded the city as the market. We need to change this.

That said, going back to the Venezuelan share measurement, it is currently at the school level.

4. Goal

- Fix the above, if necessary
- Replace the student heterogeneity term to be high achievers and Venezuelans
 - Gender is not an exciting heterogeneity in this context
 - Since Venezuelan must have different taste to their own race and Peruvians, and it affects equilibrium sorting, this is a very relevant heterogeneity
- New specification: consider pupil-teacher ratio as another characteristic, and we will find a new IV for it
 - For similar terms in the demand, one can check
 - Incentivizing Better Quality of Care: The Role of Medicaid and Competition in the Nursing Home Industry
 - Angela Crema's job market paper

AutoSave 20260127 new structural — Saved to my Mac

Home Insert Draw Design Layout References Mailings Review >> Comments Editing Share

Paste Aptos (Body) 12 A⁺ A⁻ Aa A₀ A₁ Paragraph Styles Dictate Add-ins Editor Copilot

iii. Change market specification to be grade-city

1. Let's try all our previous specifications and compare. That means compare
 - a. Original specification with and without this sample change
 - b. New specification (with V as the heterogeneity) with and without this sample change.
 - c. Do some summary stats' check by grade

iii. Results

1. See city_grade_results.pdf

6. New

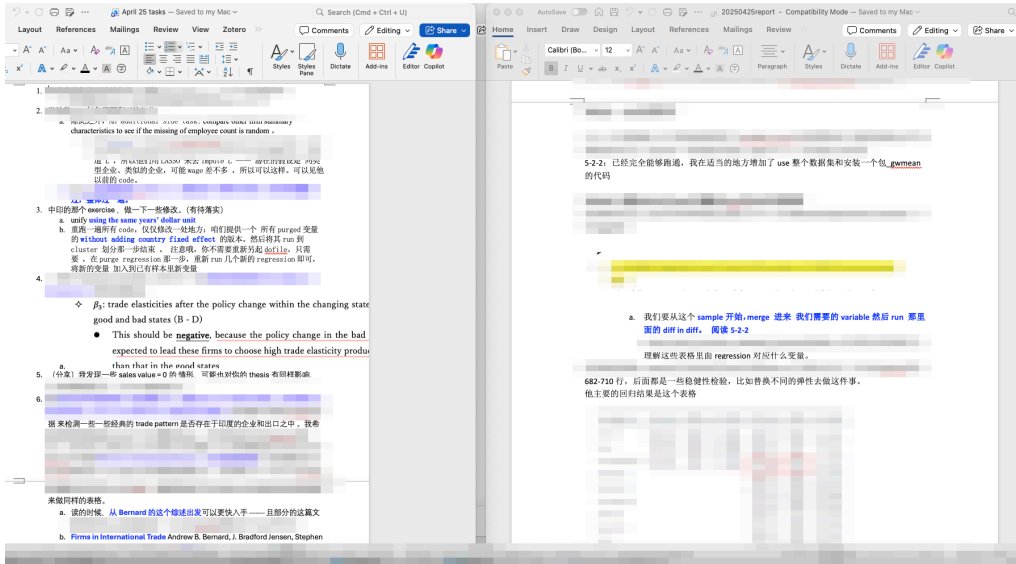
- i. + add PT ratio
- ii. + try different IV for PT ratio
- iii. + try and try and try until we clearly understand the data variation and which IVs work and which do not. By the way, one can not understand how to tune parameters if she knows the data variation well, so at some point, you need to work back and forth about structural estimates and data summary stats. Do get yourself familiar with it.
- iv. counterfactual

7. Please keep a worklog for both of us, on what you did each time when you decide to do the RA work. I will put what I did on it, too. It seemed costly, but let me tell you this: it is inevitable! Say you want to try 4 different specification on Jan 30. Believe me, you will forget what exact specifications you were trying on Jan 31. Keeping a logs will allow you to monitor the control experiment of trying specifications/samples.

20260205 update on file location (Faqiang)

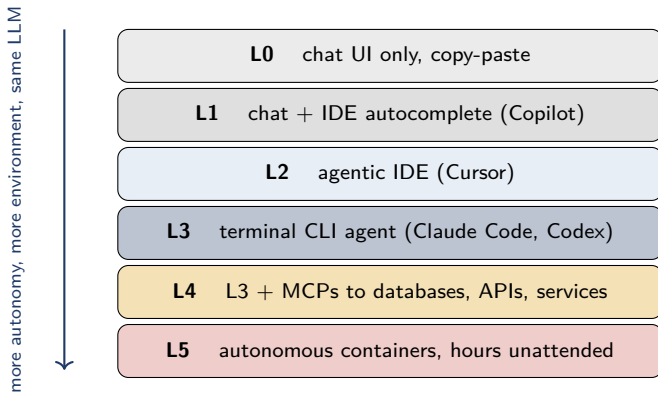
Faqiang has sent through wechat from dropbox

Page 5 of 17 2697 words English (United States) Focus 145%



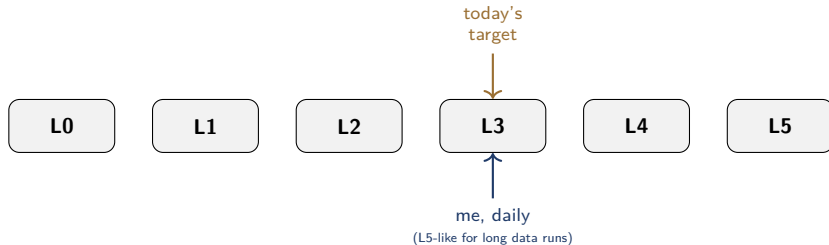
A task list and an RA's report side by side. Only the names, the data sources and one unpublished table are masked — the point is the length and the structure.

Part a: The Kyle Jensen mastery ladder



Every level runs the same underlying model. You are only adding tools and structure around it.

Part a: Where I am, and where today gets you



Me

- L3 every day, and **mainly the CLI** — Claude Code in the terminal, not the desktop app, not an IDE.
- L5-like only for data collection: runs of 10–30 hours with many parallel AI workers and no one watching.
- Still weak where my own look-back says I am weak: git, project rules, deny rules (Part e).

Part b

Claude Code basics, fast and live

Part b: Words you will hear today

Model the language model (Opus, Sonnet, ...) that writes text and commands.

Harness the program around the model that feeds it context and runs its tools. **Claude Code is a harness.**

Agent model + harness that can act: read files, run commands, search, keep going until done.

Context window everything sent to the model in one request.

Session one conversation, saved as a `.jsonl` file on your disk.

Plan mode it reads and proposes; it changes nothing until you approve.

/plan

Today is about Claude Code — its cousins (Codex, Gemini CLI) mostly work the same way; I will say so when something is Claude-specific. **From here on I call it cc.**

Sub-agent a helper with its own fresh window that returns a short result.

Skill packaged instructions loaded only when needed.

Memory, CLAUDE.md files loaded at the start of a session: rules, notes.

Shell, bash the program that runs typed commands in the terminal.

MCP a standard plug for outside tools and data — this is what lets Claude Code drive Stata or a database for you.

Part b: My daily routine, live LIVE

When	What I type	My habit
Starting a task	<code>cd <project>; claude -n "[Project] task: steps"; /model; /effort; /plan</code>	every session, in this order
Picking work back up	<code>claude --resume (the picker); /rename</code>	/rename the moment a session turns out to be about something else
Before a big step	<code>/context; /usage</code> $< 80\%$	/context before anything long or expensive
Handing over to the next session	<u>a handoff file</u> the next session starts from	written at a task boundary, never at the end of my patience
Leaving it running	<u>/workflow; /loop; /remote-control</u>	mostly automatic once started

Two things I set up once, beyond opening a session:

- **An MCP for Stata and for MATLAB**, so cc can run them directly instead of me copying commands across.
- **Permissions and deny rules**, so it never has to ask about the routine things and can never touch the dangerous ones.

Live on a demo folder with made-up data. Run sheet: `live/runsheet_b_basics.md`.

Part b: The one concept the rest of today is built on

Definition

Everything you and Claude exchange in a session — your messages, Claude's replies, every file Claude reads, every command output, every tool call — gets bundled together **every turn** and re-sent to the model. That bundle is the **context window**.

Three concrete facts:

- The window has a fixed size. For Claude it is roughly **1 Million tokens** ($\approx$ 750,000 English words, $\approx$ a 1500-page book).
- Every turn re-sends the *whole* window. The model does not “remember” — it re-reads.
- Whatever you put in the window competes with everything else in the window for the model's attention.

Each turn is not chat. It is more like *passing a very long document back and forth*.

Part b: What actually lives in the window

Anatomy of an LLM Chat

13 / 26

```
<|start|>system<|message|>You are ChatGPT, a large language model trained by OpenAI.  
Knowledge cutoff: 2024-06  
Current date: 2025-06-28  
Reasoning: high  
# Valid channels: analysis, commentary, final.  
Channel must be included for every message.  
Calls to these tools must go to the commentary channel: 'functions'.<|end|>
```

system prompt
(written by openAI)

```
<|start|>developer<|message|># Instructions  
Be concise and technical. Provide code examples when relevant.
```

developer
message
(tool definitions)

```
# Tools  
## functions  
namespace functions: {  
  type web_search = {_: {  
    query: string,  
    num_results?: number,  
  }} => any;  
} // namespace functions<|end|>
```

```
<|start|>user<|message|>What's new in the latest version of polars?<|end|>
```

user
prompt

```
<|start|>assistant<|channel|>analysis<|message|>User asking about latest polars version.  
My knowledge cutoff is 2024-06, need to search for recent updates.<|end|>
```

thinking

```
<|start|>assistant<|channel|>commentary<|message|><|call|>functions.web_search  
{  
  "query": "polars python latest version new features", "  
  "num_results": 5  
}<|end|>
```

tool call

```
<|start|>tool<|message|>[{"title": "Polars 1.8.0 Release Notes", "snippet": "New  
streaming engine, improved join performance, added pivot_wider...", "url": "..."}]<|end|>
```

tool
results

```
<|start|>assistant<|channel|>final<|message|>Polars 1.8.0 (latest as of Nov 2025)  
includes:
```

```
- **Streaming engine overhaul**: Better memory efficiency for large datasets  
- **Join performance**: 2-3x faster for certain join types  
- **New API**: pivot_wider() for reshaping data  
- **LazyFrame improvements**: Enhanced query optimization
```

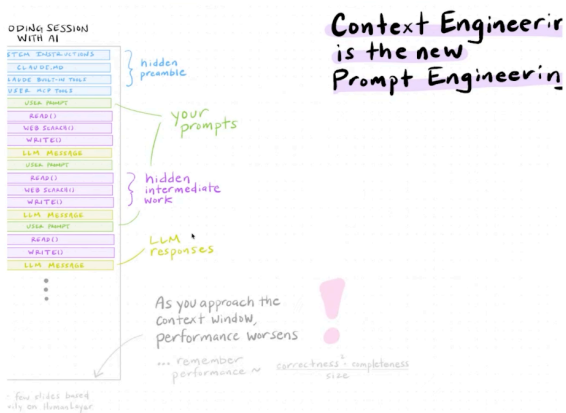
final response

```
Check the release notes for migration details if upgrading from older versions.<|return|>
```

ANATOMY OF A
MODERN LLM
CHAT

You
typically
see only
these
two!

The model can not reason well if it has approached its context limit.



Everything I do in a session is an answer to this one limit.

Part b: A session is a notebook on your disk

- Every session is saved as one file:
~/.claude/projects/<folder>/<session-id>.jsonl
- Think of the file as a **notebook** and each turn as a **page**.
- Clearing the context turns to a fresh page. It does not throw the notebook away — the agent just stops reading the old pages.
- `claude --resume` reopens a notebook. A good title (`claude -n "[Tag] task"`) is how you find it again.
- The folder name comes from where you started `claude`.
Start inside the project folder, not your home folder.
- **This pile is worth pruning and worth reading.** How I do both — and what six months of my own notebooks said about me — is Part e.

Try it (second terminal)

```
ls -lh ~/.claude/projects/  
wc -l <newest>.jsonl
```

LIVE shown on the demo folder only

Part b: One example runs through this part — how I prepared this workshop

The big task: turn six months of using cc into a two-hour workshop. Far too big for one session. This is how it became a set of them.

0. **The context I already had**, from the June version of this talk at PKU: my notes on GP's videos, a four-page Word file of tips jotted down over six months, a product manager's guide to Claude Code, and Gentzkow and Shapiro's *Code and Data for the Social Sciences*.
1. **Start from the June framework**. It already had the context-engineering rules, so one session turned it into a .md skeleton. What it lacked was examples. Where would they come from?
2. **A six-month review of my own sessions**, run as its own project: read everything back, distil use cases and lessons — for the slides, and for me.
3. **Finish GP's videos**: record my reactions while watching, screenshot, and hand the captures to a separate session to collect whatever is new.

The next three were not planned. Each arrived under one rule: when the context fills, summarise and switch.

4. **Privacy**. Some of my research could not be shown as it was, so a session was given the rules and masked the pictures.
5. **Two iterations across sessions**. The first draft was followed by heavy deletion; deleting produced better logic and better use cases, twice.
6. **Hour 2 needed a session of its own**: the structural projects had too many candidate moments, so extracting them became a separate job.

Part b: Live — planning a set of sessions, and the prompt that opens one

LIVE

Both demos are about one thing: **planning ahead, and predicting what the agent will need to know.**

Live 1 — planning this workshop

The example on the previous page, in the sessions themselves: how one big task became a set of sessions, and the prompt that opened the first of them.

Live 2 — a small task, written well

A recent task in the solar coordination project: constructing an exposure index. One prompt, read clause by clause — what it fixes in advance, and why.

LIVE

Two sessions: the one that planned this workshop, and the exposure-index task in the solar coordination project.

Part b: Why the prompt is the work

This is not an AI skill. Deciding what the task actually is, what it rests on and how you would know it worked — that is research ability. The agent only makes the cost of being vague visible sooner. **Writing the brief**

- **Sketch first, then let it refine.** A perfect prompt is not possible — there is always something you do not know you do not know.
- **Let it interview you:** *“Before you implement anything, interview me: ask the questions whose answers would change what you build. One round, then wait.”*
- **Be specific.** “Compute monthly growth by sector, BLS-style formatting” beats “analyze this data”.
- **Front-load the specifics:** inputs, outputs, constraints.
- **Say what must not happen:** files not to touch, things not to build, what to leave for later.
- **Say how you will know it worked,** when you can: a number to reproduce, a count to match, a place to stop.

LIVE *The session that prepared this workshop: the window filling, what went to a file instead of the chat, and where I stopped.*

Part b: Inside one session — the whole rationale

grep.

What the window does to you

- **Big files crowd out everything else.** “Read this 30K-line log” spends the budget your reasoning needed.
- **Long sessions crowd out their own beginnings.** What you settled at turn 2 is hazy by turn 50.
- **Start a new session by the time the window is 80% full.** Not when it breaks — by then it cannot even write its own handoff.

Other habits to follow

- **Iterate on the output, not the prompt.** “Move the legend top-left”, not the whole request again.
- **Correct early.** Wrong at turn 2, fixed at turn 3 — not turn 10.
- **Trust but verify.** Check the boring cases and the edge cases.
- **Break work into chunks:** 5–10 focused turns, one objective.
- **Write state to disk.** Decisions and progress go to files, not into the chat.
- **Research-heavy detours belong in a sub-agent, not in the thread you are working in.** Browsing five pages, comparing four packages: *“spawn a sub-agent to investigate X and bring back a short summary.”*
- **Take exact numbers off the saved files,** not off the conversation. A long session misremembers its own output.

LIVE *The workshop's own ProjectManagement/ folder: the handoffs, the plan copies, and the summary that opened the next session.*

Part b: Across sessions — the whole rationale

Four triggers to start a new session

- ① A small task is finished. The next task is the next session.
- ② You are about to branch a risky version.
- ③ You want to run two similar things side by side.
- ④ You want to pilot something small, then scale.

The leave-behind ritual, written *before* you close

- ① A summary of what was done.
- ② A paste-ready starter prompt for the next session.
- ③ One educative note for the project's KNOWLEDGE.md.

Without all three the next session pays the same context tax twice.

What the handoff must contain to be worth writing: frozen facts,

real file paths, what is still open, what *not* to redo, and a checkable bar for done. **A decision made in a picker leaves no**

trace. Anything chosen through a menu or an approval prompt is invisible to the next session — restate it in plain text.

Laptop level

- **Always name a session:** `claude -n "[Tag] what changes"`. One tag per project, plus `[temp]`.
- `[temp]` sessions are the right substitute for opening a chat tab.
- **Several terminals at once** — one task each. The cheapest parallelism there is.
- **Two writers on one folder will collide.** One writer per folder, always.
- `"cleanupPeriodDays": 730` — old sessions survive long enough to be reviewed.

Project level

- **A CLAUDE.md per project:** what it is, which folders are read-only, which script is the one that runs, what “done” means. It loads when you start inside the folder.
- **Skills** for anything you repeat.
- **One git repository per project; data stays out.** Sessions cycle and lose context — git does not.
- **A ProjectManagement/ folder:** handoffs, plans, meeting notes, issues, checks.
- **A memory rule written from one example has no scope until you give it one.**

Part c

Hacks worth stealing

Part c: One-liners — steal these first

Four lines that change what you get back. No screenshot needed; the phrasing is the whole thing.

- ① *“Take your time. I know you are fast. I need quality and verification.”*

Slows it down when the work needs care rather than speed.

- ② *“Use sub-agents to parallelise.”*

The most counter-intuitive one. Use it whenever you were about to read three things in a row — and whenever you catch yourself browsing inside your working session.

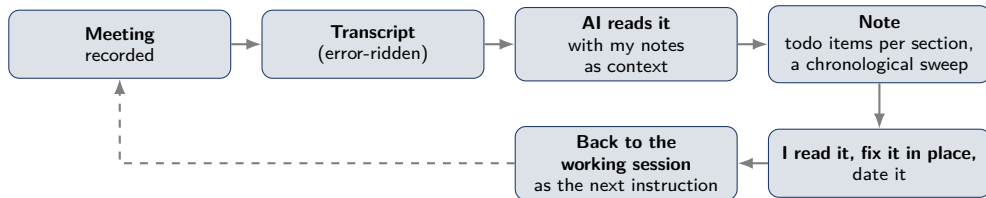
- ③ *“Before you implement anything, interview me: ask the questions whose answers would change what you build. One round, then wait.”*

Cheaper than trying to write the perfect prompt, which is not possible anyway.

- ④ **Ignore its adjectives.** *“The cleanest version yet”, “a great catch”* — praise is not evidence.

When it admires its own output, tell it to argue the other side and read that instead.

Part c: The meeting routine — from a recording to the next instruction



The key part of my prompt — the transcript and my own meeting notes are attached

Summarize and group discussion contents in sections and the todo items for each section and in the end provide a chronological sweep of the things went through in the meeting. The transcript is deemed to be error ridden so the context you can base on in translating the wrong words can largely come from the notes I supply. But the summary meeting note you give me should just base on the meeting per se. Also, no irrelevant chit chat content should be included.

- My notes are the **context** that repairs the transcript's wrong words; the summary itself must come **only from the meeting**.
- Paste your own doubt in with the note, so the agent weighs the coauthor's proposal instead of obeying it.
- GitHub issues are the better carrier for the todo items. This is how to do it without them.

 Welcome, faqiang

new coordination [Speaker 3] OK,那我们就开始吧,我把那个note发给大家,我那个note上没有什么新内容,就是要跟大家讲我们上次讨论的东西....

PASTED

PDF

PDF

We had a meeting and this is the meeting transcript. I presented this meeting note destination_exposure_note_detailed for the coauthor to showcase their strength, and we continue to discuss more about the identification problems and solutions shown earlier in this note: identification_note. There are some valuable new points raised from the group discussion. Summarize and group discussion contents in sections and the todo items for each section and in the end provide a chronological sweep of the things went through in the meeting. The transcript is deemed to be error ridden so the context you can base on in translating the wrong words can largely come from the notes I supply. But the summary meeting note you give me should just base on the meeting per se. Also, no irrelevant chat content should be included.

+ Chat Cowork

Opus 5 Max   

Meeting summary with discussion sections ...

and who owes what.

The pivot in section 8 is the one I'd flag as the meeting's real output. Once the location term is a fixed effect rather than an error, most of the exclusion discussion in sections 5-7 is defending an instrument against a problem that isn't there, and the live threat moves entirely onto size being observed only at the chosen location.



Coordination meeting summary
Document · MD

Download

    just now



Coordination meeting summary · MD Copy  

Coordination meeting — summary note

Speaker labels follow the transcript (S1, S2, S3). S2 presented the destination-exposure note and the new data; S3 opened with the identification recap.

1. Where the size shifter sits in the identification strategy

Recap of the problem carried over from the previous meeting. If investment size is endogenous, in the sense that a firm's size depends on its unobserved cost, then rivals' total landed capacity at a location is endogenous too, and it needs an instrument.

The agreed shape of that instrument is a product of two factors: the predicted probability that a rival firm lands at location i in period t , times a firm-

The prompt with the transcript and my notes attached, and the note that came back.

Part c: A weekly literature feed you pick from

Task. Every week: collect the new working papers and journal articles in my field, drop what I have already seen, sort by topic, and let me file the ones I pick into the reference manager. Built once, in plan mode; now one command.

The opening prompt — the part that defines the product

Input: weekly feed from nber working papers + top 5 journals + Rand (because I am an empirical IO economist) + AEJ applied + IER + REstud.Output and how I expect the outputs to be used: 1. I can read weekly a feed of all the empirical IO (no need to include theory papers) papers with their abstract and one sentence brief, categorized and delivered to me2. After scanning them, I will pick some to download so that I can read further3. the downloaded files should be wired to Zotero system and stored properly(the skimmed papers should not be added to the Zotero) 4. if any of the above steps are modified later, the thing we built should easily adapt5. anything else you think are useful to addThink about it and return it with my plan.

Effect. A feed page every week that I skim and tick, instead of newsletters I do not read; the picked papers land in the right Zotero collection. It is also a reading partner: point it at what it just collected, rank them, ask what each one would change.

Part c: Confirming what the literature actually says

Task. A note was drafted with references. Before it went anywhere, check that the papers say what they were cited as saying.

The prompt

double check again your reference to see if the it appears indeed in those papers

Effect. Problems in **all six** references: an invented verbatim quotation, swapped editors, wrong journal details, a wrong paper, and a false claim about a real one.

So make it standard, not occasional

Require **the paper and the page** for every claim — “cite the page where this is stated”. Then **open one or two at random and read them yourself**. Asking whether a paper exists is not the same as asking whether it says what you were told.

Part c: Putting your reference manager in order

Task. Sort several hundred unfiled papers into my existing folder structure, merge duplicates, and do not disturb anything already filed.

The prompt, in plan mode, with a screenshot of the folder tree attached

you are gonna access to my zotero folders and put papers into different folder, according to, say, my side left column [Image #1].
don't move the existed, already-grouped papers.

Effect. Unfiled papers fell from about 300 to a few dozen in one session. It backed the database up first and tagged every change, so the whole thing could be undone in bulk.

The transferable part

Before any bulk change to a library you care about: back up, tag every change so it is reversible, and test the rule of thumb you are about to trust. Mine — “the bigger file is the one with my notes” — turned out to be right less than half the time.

Part c: Before you trust a new dataset, replicate a published table

Task. A slim extract had just been built from seventeen years of a very large administrative dataset. Check it before building anything on it.

The prompt

I am an assistant professor of economics but have limited knowledge about international trade practice. I put a paper [paper] in the folder, read the solar panel section, table 9.3, try to replicate the numbers, if very close, great — interpret the number to me concretely with examples

Effect. The mismatches exposed **five real bugs** in the new extract — including an import/export flag that was reversed — before a single result rested on it. *Someone has already published a number from almost any dataset you are about to use.*

Reproduce it first.

Part c: “Be honest: were the results bent toward what I expected?”

Task. An overnight run had collected construction costs from about 130 firms’ filings — to check a number my own model had produced. The agent knew what I expected.

The morning question

be honest: any deliberation or selection on the results? I want the search process to be independent and trustworthy and you should not be affected by my earlier rough search estimates and my structural estimates

Effect. It searched its own prompts, found two places where my expectations had leaked in, and re-ran the numbers without them. The main result held — which is the only reason I can cite it. *Ask for the test, not for reassurance. And the cleanest version of this check is a sub-agent with no memory of what you were hoping for.*

Part c: Debug by meaning, not by syntax

Task. A result looks wrong. Do *not* open the code and start reading line by line — the cheapest debug is in plain English.

The prompt

Walk through the calculation step by step on three example rows and tell me what you find.

Effect. Compare its walk-through against your own intuition. **If they disagree, you have a target** for a code-level look. **If they agree, the problem is upstream** — in the data, not the code. *This only works if you have your own feel for the data. That is*

what all the eyeballing is for.

Part c: The N-ledger — every merge and filter prints its counts, and you read them

Task. The agent writes and runs the cleaning code, so you never see the merge table yourself. Every merge, filter and drop quietly redefines the estimating sample. **The rule that goes into every data brief:** print rows in, rows out and rows dropped after every merge and every filter — **an unexplained change in N is a bug until it is explained.**

The prompt, in the form I now use

Show me how you get to this number of observations from each of your data selection codes.



Effect. The “active” filter was doing nothing — over half the rows had zero capacity. Fixing it voided a night of estimation runs and flipped a key sign. Another time, one line of a summary — *91 rows had no match* — led to a case-sensitive merge that had silently dropped a firm from every regression of two sessions.

No regression output would have shown either. The ledger is thirty lines you can read without reading any code.

Part c: Anchor, invariant, negative — a failed gate stops the work

Task. The agent changes estimation code over months, and you will not read every diff. So name three cheap tests in advance, and have them run on every change:

- **Anchor:** reproduce a frozen number to a stated tolerance.
- **Invariant:** a quantity that must be exactly zero.
- **Negative:** break an input on purpose, and require the named error to fire.

The sentence that puts the agent inside the gates — from my own operating manual

After each change run the gates and paste the results. Stop and wait when the gates pass or when you need a decision from me.

Effect. A rebuilt panel had to match the coauthors' file with **zero difference** before anything new — and the rebuild exposed a misspelt firm name that had been dropping quarters from the old data. A brute-force likelihood matched the fast one to 6×10^{-16} . A patch transcribed into a second language and run offline caught a 0/0 before a seven-hour server run.

Part c: A context-less audit, anchored on the script that actually runs

Task. Commission an independent audit of estimation and simulation code — from a reviewer that has not absorbed the working session's assumptions.

The protocol I set

I want the audit to be context-less (hence not conditional on any prior knowledge – your request to such parallel subagent should just be, explaining function of scripts, explaining theories, and that's it), stating the theory, and check internal consistency of the estimation and simulation algorithms with all related scripts

Effect. Two audits with different framings, kept as separate reports so their disagreement stayed visible. And the three rules it left me with:

- ❶ **Line 1 of every audit brief names the live entry script** — the one you actually run. An audit not anchored on it will happily review dead code; both of mine had cited lines that were commented out.
- ❷ **A finding that cannot quote a live line is void.**
- ❸ **Two audits with different framings, never one.**

Part c: Make the agent try to break your identification argument, by simulation

Task. A coauthor's note argues that three parameters are identified. Before touching real data: build the game, simulate data from it, and estimate with the note's own routine.

My brief (excerpts)

let's simulate the location game and generate data according to the specification and then estimate it using the estimation routine in the noteto see how are we doing in recovering the primitiveswhether they are legit and any loopholes in our economic model and identification strategy in implementing it and what else assumptions/data do we need.

Effect. A six-page verdict: each argument holds in principle and is weak in practice — one parameter is identified, but its standard errors are many times the truth's spread.

A Monte Carlo of your own identification argument used to cost weeks, so nobody ran one. Now it costs an evening — and “find the loopholes” points the agent against you instead of with you.

Part d

Collecting data at scale, with two models

Part d: One operation, start to finish

The research need. For a project on coordination in the Chinese solar industry we needed firm-level investment events — which firm built what capacity, where, and when — for **1,373 solar firms**, at a granularity no database sells. The evidence exists, scattered across company filings, project permits, local government notices and the trade press. **The operation.**

Ten parallel sessions, three at a time. **Claude and DeepSeek** doing two different jobs. A machine gate between collection and extraction, and RAs checking behind it. About **\$350–420** for the full run. **Why I show it.** It is the most complete

example I have of AI doing research work at scale, and the one most of you will want to adapt. The point is not the numbers but the shape: how a prompt like this comes to exist, and how to read one. Two halves:

- ① **The journey to the prompt** — splitting the job in two, the trials that made it work, and a format an RA can check without being told how.
- ② **The final prompt, block by block** — what each part is for, and what it changed.

Then what went wrong anyway, and the outputs themselves, live. *The industry is solar, as you would guess. No firm is named; ids like P0506 are our own.*

Part d, the journey (1): Read the requirement thin, before any prompt

Four columns, and nothing else

Goal · Constraint · Decision point · Unknown. Anything you can rule on now, rule on now. Anything you cannot becomes *“the pilot answers this”*.

- **Freeze the frame and give it a version number.** Mine went 1,588 → **1,373** units once duplicates and out-of-scope firms were removed. A frame that moves during a run is not a sample.
- **Separate what we already have from what we are going out to find. Anchors are not answers** — they exist to confirm identity and to cross-check, never to be counted as findings. (1,225 of 1,373 units had an anchor; 419 of those under a former name.)
- **Cut the job in two: zero-judgment collection → judgment-bearing extraction.** Two reasons, and the second is the one people miss:
 - **Replicability.** All judgment sits in one stage. It can be audited, re-run and changed without going back to the web — and what was collected survives every change of mind about how to read it.
 - **No contamination.** A worker that is judging while it searches will search for what it has decided to find.
- **Seed the pilot with cases whose answer you already know**, mixed in so the worker cannot tell them apart. A batch that returns 0 of 10 known answers has told you something no summary statistic would have.

Part d, the journey (2): Pilot on a handful chosen to break it

25 units, deliberately not random — picked to cover the extremes. Result: **17 found, 7 with no program, 1 excluded**; 92 projects, 265 events, a 96-row panel across 40 cities. **What the pilot converted from guess to measurement:**

- **Hit rate falls monotonically with firm size:** 8 of 8 large, 4 of 5 mid, 4 of 6 small, and **0 of 5** among firms killed by the anti-dumping cases. Full-run expectation: 30–45%.
- **Where it hangs:** the biggest firms have too much to read (one closed the socket after 27 calls, another idled out after 49); the smallest have too little (one took 91 calls, another 24 minutes without converging).
- **Anchor inflation**, the failure that mattered most: one unit reported 18 of 23 projects that were really just the anchor restated. After the discipline was enforced: **5**.

Part d, the journey (3): Three pre-flight checks, every time

Before a run that costs real money, make the agent prove the tools work **in the environment the run will use**.

- ① **Can it reach the network?** Have it fetch one public URL and show you the status line.
- ② **Can it run code and write to disk?** One test file, one shell command.
- ③ **Can it get text out of a PDF?** Download one, then extract locally and show the first lines.

When a tool is blocked, run the same-machine control

`curl` inside the agent returned 403; the same `curl` in my own terminal worked. That one comparison separates *the network is blocking me* from *the sandbox is blocking me* — and they have opposite fixes. The decisive test that unblocked the run: fetch the PDF with the agent's own web tool, then extract the text locally.

Part d, the journey (4): A format an RA can check — two tables and a droplog

The second stage reads only the evidence files. It never goes back to the web.

- **Table A, one row per project:** id, name, location, category, target size, amount, time anchor, status, credibility, **the evidence id it rests on**, notes.
- **Table B, one row per dated observation:** project id, year, month, event type, value, unit, definition, **how the year was assigned** (from the text, or as-reported), evidence id, credibility.
- **The droplog:** every piece of evidence you decided *not* to use, with the reason. **If there is a number you are not using, write down why.** This is the file that answers a referee.

Four red lines, written into the prompt

No imputation. No extrapolation. An event with no date does not enter the table, and a year with no evidence is not filled in. **A target is not an outcome. An anchor is not a project.**

Part d, the journey (5): Write the quality standard as code

A rule stated in prose is a suggestion. The same rule in a validator is a gate.

- **A controlled vocabulary is a set** — anything outside it is an error, not a warning.
- **“Value first” means prefix matching.** The controlled field is read by its first token only, which is why commentary is forced into brackets.
- **Every row traces to an evidence id that actually exists** in the evidence file.
- **Consistency:** every time anchor in Table A must have its event row in Table B; dates inside the window.
- **Warn on silent omission:** a number present in the evidence and used nowhere is a flag, not a shrug.

It works: the real outputs passed, and **19 deliberately injected errors were all caught.** *Put the substantive rule you care about into the gate — not just the structure.*

Part d, the prompt (6): Block by block — the first four

Block	What it must do, and why
1. The job and its hard edges	Role, and what is out of scope. Without the edges it widens the job by itself.
2. Identity and anchors	The unit id, its given name, known aliases, and the anchor record — with the anchor discipline <i>written into the prompt</i> : anchors confirm identity and cross-check, they never become findings.
3. The evidence block	One controlled schema per fact: a verbatim quotation you could Ctrl-F on the original page, the controlled field, the number with its unit and definition, the exact URL, how it was fetched, and a one-line recipe for re-checking it. Controlled fields start with the value; commentary goes in brackets — a machine reads the first token.
4. Sources, ranked	Tiers A/B/C/D, and a trace left for every tier it touched — including the ones that gave nothing.

Part d, the prompt (7): Block by block — the last four

Block	What it must do, and why
5. Stop rules	When to give up on a unit, and when it is too early to. Both failures cost money: one burns the budget, the other fakes a null.
6. Evidence discipline	Verbatim, never paraphrased or translated. Two dates : when the source was published, and when the thing happened. Credibility A/B/C; corroboration raises it, but one top source is kept for checking and the rest are support.
7. Running it	Write to disk early and often, so an interrupted run is not a zero. One fact, one source — spend the budget on more units, not on re-confirming the same one. One independent context per unit ; concurrency capped. Progress in a RUN_LOG so a restart resumes instead of repeating.
8. Delivery	Two files per unit — one for a human, one for a machine, identical in content. It validates its own JSON before handing over. Frozen list, versioned prompt with a CHANGELOG, model id and date recorded.

Part d (8): Claude and DeepSeek, two jobs

What DeepSeek bought

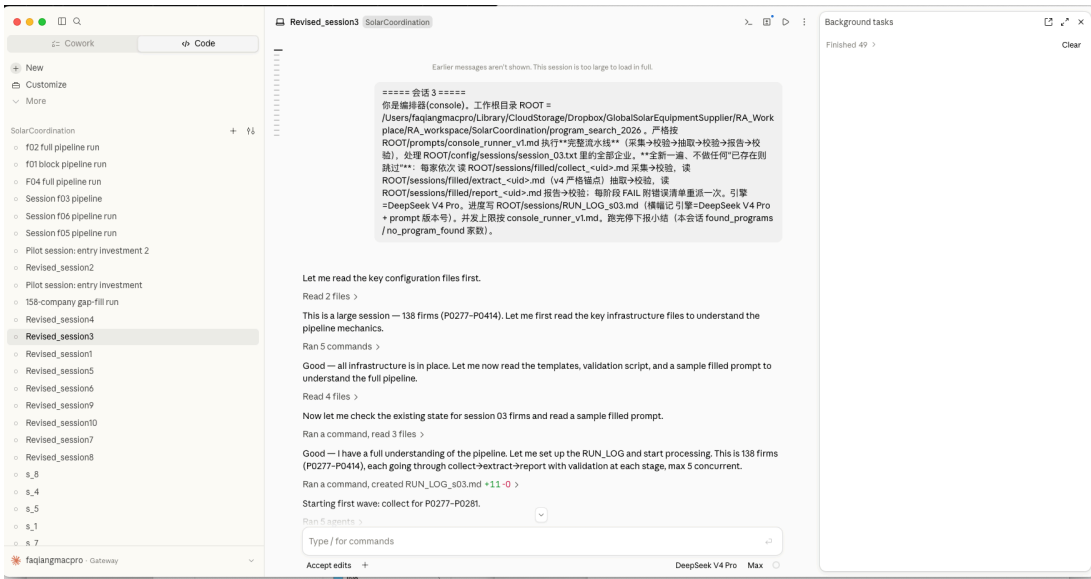
- On one large integrated maker: **42 pieces of evidence across 7 production sites**, against **19** from Claude on the same firm.
- It reached Chinese sources that Claude's sandbox blocked outright — the curl 403 from two slides ago.
- **6 firms for about \$1.50** — roughly **\$0.25 a firm**.

What it cost to run the whole thing

- Full frame: **\$350–420**.
- Six units, three at a time: 55 minutes. Median unit: 9–15 minutes.
- Ten sessions at three concurrent: **9–15 hours**.

The division that made it work

DeepSeek does the bulk zero-judgment collection where the Chinese sources are, one independent context per firm, in parallel. **Claude** runs the pipeline, the extraction stage and the gate. **The RAs** check every hit for source authenticity and faithful extraction, and blind-sample 25% of the misses. **Code** does the deterministic aggregation and deduplication — never let a model merge records by feel.



What the DeepSeek side of the operation looks like.

Part d (9): It broke anyway — and why that was survivable

The bug

The prompt was upgraded from version 3 to version 4. The runner file that actually executes the job still named version 3. Nothing failed loudly. Two units came back with a project and a dozen events each — **every one of them derived from the anchor**, which should have returned *no program found*.

Why the gate did not catch it: the validator checked *structure* — fields, vocabularies, traceable ids — and every one of those was perfectly well-formed. The rule it violated, *an anchor is not a project*, lived only in the prose of the prompt.

Two things to take:

- When you bump a version, **grep every file that names the old one** — especially the file the executor actually reads.
- **The silver lining is the architecture.** Because collection and extraction were separate and every row traced to an evidence id, the damage was found by reading, bounded in an afternoon, and repaired by re-running one stage. A single fused pipeline would have meant redoing everything.

Part d (10): Do not let the search step verify anything

You will almost always want to check a newly collected dataset against something that already exists — an official series, a commercial database, a hand-built file of your own. **That comparison is only worth running if the collection never saw the thing you are comparing against.**

How it goes wrong, concretely

I handed the workers a file of capacity figures I already owned, so they could confirm they had the right firm. The output came back about **twice as rich** as the earlier pilot — and traced firm by firm, most of the extra content was **that same supplied file, returned to me as findings**. One firm went from 26 apparent facts to a handful.

The rule that came out of it: anything you supply is an **anchor**, used only to confirm identity — it is marked as supplied, and it never counts as evidence. Otherwise your robustness check is polluted by design: you are comparing the outside source against a copy of itself.

Open the run folder: the evidence files for one firm, its Table A, Table B and droplog; then the console with three sessions running.

Part e

How to improve

Part e: Two rhythms, and that is all

Every week or two — half an hour

Have it list your sessions with titles, dates and a one-line summary of each. Close what is finished, delete the throwaways, look at what is left. And read `git log` instead of scrolling chat history.

Every few months — one deep look back

Point it at all of your own sessions and ask what they say about how you work. Not what you remember — what the files show.

Both are just prompts. They are the next two slides.

Part e: The fortnightly prompt

Task. Find out what is actually open on your laptop, and close most of it.

The prompt

I want to use this session to clean up my sessions in claude code. Given me the list of sessions, their titles, brief intro of them, established date, and active date. To get brief intro, you can go and read the first and last few sections of the session file

Effect. A table of every session you have. Close the finished ones, delete the one-liners, keep the two or three that still hold real work. **Archive before you delete** — unnamed is not empty; when I pruned 67 untagged sessions, four of them turned out to have real work in them. *Also the fastest way to find a session you only half remember.*

Part e: The deep look-back, and its brief

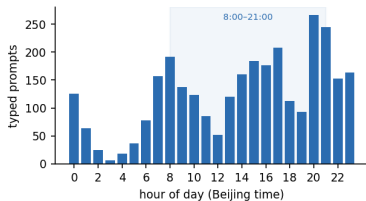
Task. Read back everything and tell me how I actually work. My first attempt returned tidy lessons anyone learns in their first hour — because the brief was vague. The second one had a brief.

The brief that made the difference

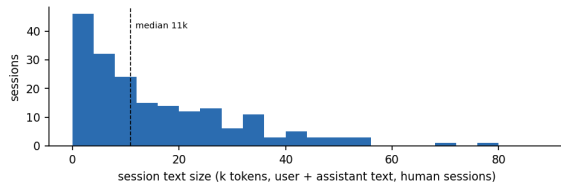
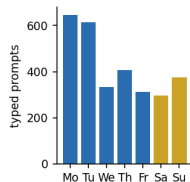
my original request was simple: what is done right, wrong (and the consequences of it), too complicated, and what should be improved. please properly group content and keep things concise. Use plain-er language

Effect. Parallel workers read 159 sessions and produced a 48-page report and 56 lessons — several of which I would have denied if you had simply asked me. ***A self-review is only as good as its brief.***

Part e: What six months of my own sessions looked like



When I type: prompts by hour and weekday.



How big my sessions actually get.

None of this is memory. Every number came from a script run over the session files.

Part e: And what the counts said

Month	Opus 4.6	Opus 4.7	Opus 4.8	Fable 5.x	Opus 5	Sonnet	Haiku	DeepSeek (fleet)
Mar	0.02	–	–	–	–	–	0.00	–
Apr	0.83	0.25	–	–	–	–	0.04	–
May	0.00	2.54	–	–	–	0.06	0.07	–
Jun	–	1.28	1.96	0.09	–	0.36	0.07	–
Jul	0.00	0.13	0.85	2.24	–	0.00	0.07	–
Aug	–	0.00	5.42	2.15	–	0.17	0.15	92.56
Sep*	–	0.00	0.68	0.74	2.05	0.38	0.01	–

Tokens by month and model family, in billions.

Tool	calls	Slash command	uses	Permission mode	sessions
Bash	62723	/plan	168	default	169
Agent	24889	/effort	156	auto	142
Edit	15896	/model	153	plan	127
Read	4340	/exit	142	acceptEdits	79
Write	2941	/compact	7	bypassPermissions	0
TaskUpdate	1959	/theme	3		
TaskCreate	993	/plugin	2		
WebSearch	440	/login	2		
WebFetch	304	/cost	1		
TaskOutput	254	/clear	1		

Most-used tools, the slash commands I actually type, and permission modes.

What distinguishes researchers now

1. **Dream big.**
2. **Verify carefully.**
3. **Still take your econ classes.**

Closing

Where to learn more

Where to learn more

- **Paul Goldsmith-Pinkham** — *Getting Started with Claude Code: A Researcher's Setup Guide*.
Substack post + two-part video series.
paulgp.substack.com/p/getting-started-with-claude-code
- **Anthropic** — regular seminars on writing good prompts.
Anthropic Academy / official changelog: anthropic.com
- **Gentzkow & Shapiro** — *Code and Data for the Social Sciences: A Practitioner's Guide*.
Research-management foundations — the rules that matter more, not less, in an AI world.
- **The first talk in this series** — *One-Stop Research with VS Code and LLMs* (handout).
zhiyuanryanchen.github.io/notebooks/handout.html